Caso de Estudio Optimización

David Restrepo
Sebastián Castaño
Miguel Alejandro Gómez

Universidad EAFIT
Bases de Datos Avanzadas

Edwin Nelson Montoya
22/02/2026

1. Objetivo

Evaluar el desempeño del sistema EAFITShop, un e-commerce, con carga OLTP y reporting. Establecer una línea base con EXPLAIN y aplicar mejoras en el sistema. Comparar los resultados antes y después de las mejoras establecidas.

2. Descripción de Caso

EAFITShop es un sistema de comercio electrónico OLTP con entidades como clientes, productos, órdenes y pagos, además de soportar analítica y reportes. El problema aparece cuando las consultas de los reportes ejecutan Seq Scan y agregaciones sobre tablas grandes, lo que degrada el motor transaccional y eleva la latencia.

3. Situación Real

Este caso es presentado por Calimatic, una empresa tecnológica que ofrece desarrollos, automatizaciones y análisis de datos. En este, se reporta la optimización de una base de datos PostgreSQL con más de 50M registros, donde un dashboard de analítica era prácticamente inutilizable. Los problemas que tenían estaban relacionados con tiempos de consultas y utilización de CPU. La CPU estaba constantemente a más del 95% y tenían tiempos de más de 60 segundos en las cargas de los dashboards, tiempos de más de 38 segundos en las analíticas de las ventas y tiempos de más de 45 segundos en los reportes de actividades de usuarios.

La solución que aplicaron fue poner logs a las queries que estaban generando estos tiempos. A partir de esto, crearon los índices necesarios para reducir los tiempos. Los índices que crearon fueron enfocados en las columnas que se utilizaban 'WHERE', 'JOIN' y 'ORDER BY', además de priorizar índices compuestos en las columnas más utilizadas. También, reescribieron las queries que generaban grandes tiempos, evitaron patrones N+1 y reemplazaron JOINs. Por último, implementaron performance tuning del servidor y generaron particionamientos de tablas de más de 10M de registros, realizándolo por fecha o ID.

Con estas soluciones, lograron una reducción de 45 segundos a 200 ms en las consultas de los reportes de actividad de usuarios, una reducción de 38 segundos a 150 ms en las consultas de analíticas de ventas, una reducción de 60 segundos a 800 ms en las cargas de los dashboards y una reducción de 95% a 25% de utilización de la CPU. En general, en tiempos de cargas, se mejoró un 99% y un 73% de utilización de CPU. Estos cambios implican una mejor experiencia de usuario y mayor capacidad de escalabilidad sin escalar hardware.

4. Ambiente Tecnológico

Se utilizó una instancia EC2 con PostgreSQL dockerizado y una instancia de AWS RDS PostgreSQL. En ambos casos se utilizó herramientas de medición como EXPLAIN y EXPLAIN ANALYZE.

5. Datos Antes de Optimizar

EC2

Query	Tiempo (ms)	Conclusión
Q1	3424.861	En la línea base se ejecuta un Parallel Seq Scan sobre orders y customer y luego se construye un Parallel Hash Join, porque no hay índices útiles para restringir por order_date ni para acelerar el join. El tiempo de ejecución está dominado por I/O (shared read=53800) y por el procesamiento de cientos de miles de filas del año filtrado. El Sort final no representa costo significativo

Q2	41660.617	La consulta está dominada por el escaneo completo de order_item (Parallel Seq Scan) y el Hash Join masivo con la tabla product, lo que genera un alto costo de I/O y procesamiento sobre millones de filas. La agregación y el ordenamiento final no tienen tanto impacto en comparación, ya que operan sobre un conjunto ya reducido para retornar solo 10 resultados
Q3	229.022	A la consulta Q3 le hace falta un índice apropiado: aunque retorna pocas filas, PostgreSQL realiza un Parallel Seq Scan sobre la tabla orders para localizar las órdenes del cliente, descartando millones de registros. Esto genera alto I/O (shared read=41103). Posteriormente se aplica un Sort por order_date DESC y Gather Merge para combinar resultados, pero estas etapas son secundarias frente al costo dominante del escaneo masivo
Q4	1.420	PostgreSQL ejecuta un Seq Scan sobre product porque el predicado ILIKE '%42%' no tiene un prefijo fijo y no puede aprovechar un índice B-tree. El motor evalúa el patrón fila por fila, lo cual se evidencia en Rows Removed by Filter: 2375. Aunque el tiempo fue bajo y los accesos fueron solo a caché (Buffers: shared hit=22), este patrón no escala: a mayor volumen de productos o menor selectividad del texto, el costo del escaneo crece linealmente.
Q5	667.940	PostgreSQL ejecuta un Parallel Seq Scan sobre orders porque el filtro aplica una función sobre la columna (date_trunc('day', order_date)), lo que impide que el optimizador use búsqueda por rango con un índice tradicional sobre order_date. Como resultado, el motor evalúa la condición fila por fila, descartando la mayoría (Rows Removed by Filter: 1665748 por worker) y realizando un volumen alto de lecturas (Buffers: read=39411). Esto domina el costo total y explica el tiempo de ejecución observado
Q6	7052.471	El plan está dominado por escaneos completos en paralelo sobre las tablas payment y orders, seguidos de un Parallel Hash Join por order_id. La tabla payment debe leer millones de filas y descartar una gran porción (Rows Removed by Filter: 1000462 por worker) porque no existe un acceso directo eficiente al subconjunto APPROVED, lo que genera alto I/O (Buffers: shared read=36370). En paralelo, orders también se escanea casi por completo (Buffers: read=38283) ya que la consulta no aplica filtros sobre esta tabla. Posteriormente, el Hash Join y la agregación por status consolidan los resultados. El ordenamiento final es marginal (solo 25 kB de memoria), por lo que el cuello de botella real es el volumen de datos leídos y el join masivo, reflejado en el tiempo total de ejecución
Q7	19446.798	La consulta está dominada por un Parallel Seq Scan sobre order_item, que obliga a leer millones de registros completos antes de poder aplicar los joins, generando un alto costo de I/O (Buffers:

		read=203200). Aunque la tabla orders sí utiliza un índice sobre order_date mediante un Bitmap Index Scan, este filtro no logra reducir el volumen de datos en order_item, por lo que el motor termina procesando una gran cantidad de filas innecesarias. Posteriormente se ejecutan dos Hash Join (orders–order_item y product–order_item) y una agregación paralela que consume bastante memoria (hasta ~43MB). El ordenamiento final es eficiente y no representa el cuello de botella. En resumen, el costo principal proviene del escaneo masivo de order_item y del join sobre grandes volúmenes de datos, lo que limita el impacto del filtrado por fecha
Q8	7740.361	PostgreSQL ejecuta un Seq Scan sobre orders porque no existe un índice que permita restringir eficientemente el rango por order_date, lo que obliga a leer millones de filas y descartar la mayoría (Rows Removed by Filter: 4,000,818). Luego realiza un Seq Scan sobre customer y construye un Hash Join por customer_id. Finalmente, la consulta queda dominada por el HashAggregate para agrupar por cliente, el cual excede la memoria disponible y genera operaciones temporales en disco (Batches: 5, Disk Usage: 19888kB; temp read/written), antes del Top-N Sort para retornar las 50 filas finales.

AWS RDS

Query	Tiempo (ms)	Conclusión
Q1	4643	Esta primera query la corrimos varias veces una con la base de datos en frío y otra cuando ya la habíamos manipulado recientemente gracias a esto obtuvimos un tiempo de ejecución en frío con el más alto de 6153.565 ms, en caliente tuvimos un tiempo de 4049.763 ms asociamos esta diferencia a que para la primera medición en frío acababamos de iniciar la base de datos, en cambio cuando lo hicimos por segunda vez ya habíamos manipulado la base de datos recientemente, lo que produjo un aumento en la cantidad de información que tuvo que ser leída en disco a diferecencia de la rapidez que proporciona poder leerla desde el cache, hicimos un tercer intento para tener un dato más aproximado a la realidad y fue de 3728 ms dándonos un promedio de de tiempo de ejecución. En la línea base se ejecuta un Parallel Seq Scan sobre orders y customer y luego se construye un Parallel Hash Join, porque no hay índices útiles para restringir por order_date ni para acelerar el join. El tiempo de ejecución está dominado por I/O (shared read= 12.469 ms) y para la otra medición en frío fue de (shared read= 2069.167 ms) y por el procesamiento de cientos de miles de filas del año filtrado, cuando la base de datos está en frió el tiempo de lectura representa un cuello de botella mucho más grande para el tiempo de ejucución mientras que cuando está en caliente es

		principalmente de cómputo. El Sort final no representa costo significativo.
Q2	17235.5	La consulta está dominada por el escaneo completo de order_item (Parallel Seq Scan) y el Hash Join masivo con la tabla product, lo que genera un alto costo de I/O y procesamiento sobre millones de filas. La agregación y el ordenamiento final no tienen tanto impacto en comparación, ya que operan sobre un conjunto ya reducido para retornar solo 10 resultados. También fue conrrido en claiente y en frío, dando resultados de 16602.829 ms y 17642.509 ms respectivamente
Q3	1203.294	A la consulta Q3 le hace falta un índice apropiado: aunque retorna pocas filas, PostgreSQL realiza un Parallel Seq Scan sobre la tabla orders para localizar las órdenes del cliente, descartando millones de registros. Esto genera alto I/O (shared read=3175). Posteriormente se aplica un Sort por order_date DESC y Gather Merge para combinar resultados, pero estas etapas son secundarias frente al costo dominante del escaneo masivo.
Q4	1.605	PostgreSQL ejecuta un Seq Scan sobre product porque el predicado ILIKE '%42%' no tiene un prefijo fijo y no puede aprovechar un índice B-tree. El motor evalúa el patrón fila por fila, lo cual se evidencia en Rows Removed by Filter: 2375. Aunque el tiempo fue bajo y los accesos fueron solo a caché (Buffers: shared hit=22), este patrón no escala: a mayor volumen de productos o menor selectividad del texto, el costo del escaneo crece linealmente.
Q5	1604.604	PostgreSQL ejecuta un Parallel Seq Scan sobre orders porque el filtro aplica una función sobre la columna (date_trunc('day', order_date)), lo que impide que el optimizador use búsqueda por rango con un índice tradicional sobre order_date. Como resultado, el motor evalúa la condición fila por fila, descartando la mayoría (Rows Removed by Filter: 1665748 por worker) y realizando un volumen alto de lecturas (Buffers: read=39411). Esto domina el costo total y explica el tiempo de ejecución observado
Q6	7052.471	El plan está dominado por escaneos completos en paralelo sobre las tablas payment y orders, seguidos de un Parallel Hash Join por order_id. La tabla payment debe leer millones de filas y descartar una gran porción (Rows Removed by Filter: 1000462 por worker) porque no existe un acceso directo eficiente al subconjunto APPROVED, lo que genera alto I/O (Buffers: shared read=36370). En paralelo, orders también se escanea casi por completo (Buffers: read=38283) ya que la consulta no aplica filtros sobre esta tabla. Posteriormente, el Hash Join y la agregación por status

		consolidan los resultados. El ordenamiento final es marginal (solo 25 kB de memoria), por lo que el cuello de botella real es el volumen de datos leídos y el join masivo, reflejado en el tiempo total de ejecución.
Q7	23830.618	La consulta está dominada por un Parallel Seq Scan sobre order_item, lo que obliga a leer millones de registros y concentra el costo en I/O. Aunque orders utiliza un Bitmap Index Scan sobre order_date, este filtro no reduce el volumen procesado en order_item, por lo que el motor sigue ejecutando Hash Join sobre conjuntos muy grandes y una agregación paralela con alto consumo de memoria. El ordenamiento final no es el cuello de botella; el costo principal proviene del escaneo masivo de order_item y de los joins sobre grandes volúmenes de datos, lo que limita el impacto del filtrado por fecha.
Q8	5608	En la primera corrida la consulta tardó ~5.61 s (Execution Time 5608 ms), lo que indica que el plan estaba haciendo más trabajo total para calcular y ordenar los resultados

6. Datos Después de Optimizar

EC2

Query	Tiempo (ms)	Conclusión
Q1	3210	Se reduce el I/O evitando acceso al heap, pero el volumen de datos sigue siendo alto (~1M filas), lo que termina por limitar la mejora. El cuello de botella no es el acceso, sino la cantidad de datos agregados y el JOIN.
Q2	21600	La optimización más efectiva fue reducir el volumen antes del join. Los índices tienen poco impacto debido a la ausencia de filtros.
Q3	0.085	Mejora drástica ya que el índice compuesto permite evitar el sort y aplicar el LIMIT directamente sobre el índice.
Q4	1.364	Aunque se implementó un índice adecuado para este tipo de búsqueda, PostgreSQL decide no utilizarlo debido al tamaño reducido de la tabla y la baja selectividad del patrón. Sin embargo, al realizar pruebas adicionales con un volumen mayor de datos, el optimizador sí optó por utilizar el índice, evidenciando una reducción significativa en el tiempo de ejecución.
Q5	3.687	La reescritura permitió aprovechar el índice que se había implementado al optimizar la query 1(idx_orders_order_date_inc), reduciendo drásticamente el tiempo de ejecución.
Q6	7070	El índice parcial mejora de forma clara el acceso a payment (menos lectura y sin acceso al heap), pero el tiempo total no cae drásticamente porque el cuello de botella permanece en orders: el plan sigue usando Parallel Seq Scan sobre millones de filas y un Parallel Hash Join masivo. Al no existir filtros selectivos sobre

		orders, el optimizador sigue prefiriendo escanear orders en paralelo en lugar de hacer millones de búsquedas por índice.
Q7	2740	Conclusión: La estrategia permitió reducir drásticamente el volumen de datos procesados, pasando de un enfoque basado en escaneo masivo a uno basado en accesos indexados selectivos. El optimizador ahora primero identifica las órdenes relevantes mediante el índice por fecha y luego recupera sus ítems asociados de forma eficiente.
Q8	7591.230	El plan dejó de hacer Seq Scan sobre orders y pasó a Bitmap Index Scan + Bitmap Heap Scan, reduciendo el trabajo de lectura para ubicar las órdenes del 2023. Sin embargo, la mejora en tiempo total fue leve porque el costo dominante sigue estando en el Hash Join con customer (que aún se escanea completo) y, sobre todo en el HashAggregate que agrupa ~632k clientes y continúa usando múltiples batches con archivos temporales (temp read/written), por lo que el índice optimiza el filtro de orders pero no elimina el cuello de botella principal (join + agregación).

AWS RDS

Query	Tiempo (ms)	Conclusión
Q1	1616.268	Se observa una reducción del I/O al evitar accesos al heap, pero el volumen de datos procesados sigue siendo elevado, lo que limita la mejora obtenida. En este escenario, el cuello de botella no está en el método de acceso, sino en la cantidad de datos agregados y en el costo del JOIN.
Q2	14864.529	La mejora más significativa se logró al reducir el volumen de datos antes del JOIN. En este caso, los índices aportan un beneficio limitado debido a la ausencia de filtros selectivos que permitan restringir el conjunto de filas procesadas.
Q3	15.922	Se observa una mejora drástica, ya que el índice compuesto permite evitar el ordenamiento y aplicar el LIMIT directamente sobre el índice, reduciendo de forma significativa el costo de ejecución.
Q4	1.567	Aunque se implementó un índice adecuado para este tipo de búsqueda, PostgreSQL no lo utiliza debido al tamaño reducido de la tabla y a la baja selectividad del patrón. No obstante, en pruebas con mayor volumen de datos, el optimizador sí eligió el índice, evidenciando una reducción significativa en el tiempo de ejecución.
Q5	12.672	La reescritura de la consulta permitió aprovechar el índice existente (idx_orders_order_date_inc), lo que se tradujo en una reducción drástica del tiempo de ejecución.

Q6	4770.762	El índice parcial mejora de forma clara el acceso a payment, reduciendo lecturas y evitando accesos al heap; sin embargo, el tiempo total no disminuye de manera significativa porque el cuello de botella permanece en orders. El plan continúa utilizando un Parallel Seq Scan sobre millones de filas y un Parallel Hash Join de gran volumen, y ante la falta de filtros selectivos sobre orders, el optimizador sigue prefiriendo el escaneo secuencial paralelo frente a búsquedas masivas por índice. El índice parcial mejora de forma clara el acceso a `payment`, reduciendo lecturas y evitando accesos al heap; sin embargo, el tiempo total no disminuye de manera significativa porque el cuello de botella permanece en `orders`. El plan continúa utilizando un Parallel Seq Scan sobre millones de filas y un Parallel Hash Join de gran volumen, y ante la falta de filtros selectivos sobre `orders`, el optimizador sigue prefiriendo el escaneo secuencial paralelo frente a búsquedas masivas por índice.
Q7	34185	El índice parcial mejora de forma clara el acceso a payment, reduciendo lecturas y evitando accesos al heap; sin embargo, el tiempo total no disminuye de manera significativa porque el cuello de botella permanece en orders. El plan continúa utilizando un Parallel Seq Scan sobre millones de filas y un Parallel Hash Join de gran volumen, y ante la falta de filtros selectivos sobre orders, el optimizador sigue prefiriendo el escaneo secuencial paralelo frente a búsquedas masivas por índice.
Q8	3197.118	Después de aplicar las optimizaciones, el tiempo bajó a ~4.47 s, mostrando una mejora clara en rendimiento. El nuevo EXPLAIN evidencia que el ordenamiento por SUM(total_amount) seguía siendo un costo importante (aparece el Sort Key), pero con las optimizaciones se redujo el trabajo previo al sort y/o se ejecutó de forma más eficiente, logrando una reducción de ~1.14 s (~20%) en el tiempo de ejecución

Particionamiento

- Se implementó particionamiento por rango sobre la columna order_date, dividiendo la tabla en particiones anuales. Esto permitió distribuir aproximadamente 1 millón de registros por partición, logrando una segmentación equilibrada de los datos.
- Se identificó que uno de los principales problemas de la consulta era el alto volumen de datos procesados en la tabla payment, donde PostgreSQL debía escanear millones de registros para luego filtrar únicamente aquellos con payment_status = 'APPROVED'. Para optimizar esto, se implementó un particionamiento por lista sobre la columna payment_status, creando una partición específica para el estado 'APPROVED'.

EC2

Query	Tiempo (ms)	Conclusión
Q1	2901.499	El particionamiento por order_date sí aportó valor al permitir partition pruning (la consulta anual se ejecuta solo sobre la partición ordersp_2023), pero no fue suficiente para cambiar la estrategia de acceso cuando el filtro es de baja selectividad (un año completo). En ese escenario, PostgreSQL mantuvo un Parallel Seq Scan porque recorrer el índice no compensaba frente a leer gran parte de la partición. En cambio, cuando el filtro se vuelve más selectivo (por ejemplo, un rango mensual), el motor puede explotar el particionamiento + el índice y cambia a Parallel Index Only Scan con Heap Fetches = 0, logrando una reducción marcada del tiempo.
Q5		El particionamiento por fecha no generó una mejora significativa en el tiempo total porque la optimización dominante ya era la reescritura sargable del predicado, que permitió un Index Only Scan sobre un rango muy selectivo (≈ 2.7k filas) sin accesos al heap. Aunque con particiones se observa partition pruning el volumen de datos leído ya era bajo con el índice, por lo que el beneficio adicional del particionamiento es marginal y el tiempo se mantiene similar
Q6	6460.232	El particionamiento por payment_status sí fue efectivo para habilitar partition pruning, haciendo que PostgreSQL consultara directamente la partición paymentp_approved y evitando el escaneo completo de payment y el descarte masivo de filas. Sin embargo, el impacto global en el tiempo fue limitado porque el plan sigue estando dominado por el procesamiento sobre orders: un Parallel Seq Scan sobre un conjunto muy grande y un Parallel Hash Join costoso. En consecuencia, el particionamiento optimiza la lectura y el join desde payment (menos filas, menor costo de construir el hash y menos bloques leídos), pero no ataca el cuello de botella principal, que es el escaneo masivo de orders.
Q7	2465	Al igual que con la anterior query, el particionamiento por fecha no generó una mejora significativa en esta consulta, ya que, aunque permite restringir el acceso a la partición correspondiente (ordersp_2023), el plan de ejecución sigue dominado por el procesamiento de grandes volúmenes de datos en order_item y por los joins masivos. En este caso, el cuello de botella no está en la tabla particionada, sino en la cantidad de registros que deben ser agregados y combinados, por lo que el particionamiento no impacta de forma relevante el tiempo total de ejecución

Performance Tuning

Configuraciones Aplicadas
• work_mem = 128MB
• random_page_cost = 1.0
• effective_cache_size = 8GB
• effective_io_concurrency = 100
• max_parallel_workers_per_gather = 2
• track_io_timing = on

EC2

Query	Tiempo (ms)	Conclusión
Q1	~2900	Sin mejora significativa
Q2	~0.1	Óptima
Q3	~1.3	Sin cambio relevanta
Q4	~3-4	Óptima
Q5	~20000	Sin mejora significativa
Q6	~6400-6500	Mejora leve

Estos resultados se explican porque el cuello de botella principal no estaba en la configuración del motor, sino en la naturaleza de las consultas. En particular, varias de ellas procesan grandes volúmenes de datos (millones de filas) sin filtros altamente selectivos, lo que obliga al optimizador a elegir planes basados en Parallel Seq Scan y Hash Join, donde el costo está dominado por CPU más que por I/O o memoria. Dado que la instancia utilizada no presentaba saturación de recursos (CPU mayormente ociosa, sin presión de memoria ni iowait significativo), los cambios de tuning tuvieron un impacto limitado. En este contexto, mejoras adicionales requerirían cambios estructurales más profundos, como rediseño de consultas, uso de agregaciones precomputadas o estrategias de almacenamiento orientadas a analítica.

7. Líneas Futuras de Trabajo

Para un trabajo futuro, nos deberíamos enfocar más en cambios estructurales que en realizar tuning, porque, con los resultados obtenidos en las queries, se evidenció cómo todavía se siguen utilizando scans y joins sobre millones de filas. Por ende, se debería seguir con las vistas materializadas ya creadas y complementar preagregaciones para reportes recurrentes y refinar el diseño de índices.

8. Uso de IA

La IA se utilizó en la búsqueda de la situación real en una empresa. Se utilizó perplexity para buscar en diversas fuentes y tener un repositorio de posibles fuentes para el informe. También se utilizó para una mejor comprensión de las queries, mejor entendimiento de los resultados de las queries y posibles mejoras u optimizaciones a realizar a las queries y tablas.

9. Referencias

https://calimatic.com/blog/postgres-performance-optimization/

10. Repositorio

https://github.com/scastanoa1/BasesDeDatosAvanzadas/tree/main